

DOMAIN-SPECIFIC KNOWLEDGE ASSISTANT USING RAG

**A Minor Project Report
submitted as part of the ongoing Internship Program in
AI Solution-Driven Domain**

Submitted by

Janma A Saragodu

Under the Guidance of

Mrs. Sruthi Tarimana

Organization

Unlox Academy

Internship Duration: February 2026 – April 2026

ABSTRACT

The rapid growth of information has created a need for intelligent systems that can retrieve accurate and domain-specific knowledge efficiently. This project presents the design and development of a Domain-Specific Knowledge Assistant using the Retrieval-Augmented Generation (RAG) approach. The system is capable of answering user queries strictly based on the content of uploaded PDF documents, thereby minimizing hallucinations commonly observed in large language models.

The application integrates LangChain, Google Gemini API, FAISS vector database, and HuggingFace embeddings, with a user-friendly interface built using Streamlit. The chatbot processes documents, converts them into embeddings, retrieves relevant information, and generates precise answers. A strict prompt design ensures that responses are limited only to the provided data.

This project demonstrates how modern AI techniques can be combined to create reliable, explainable, and domain-focused conversational systems suitable for academic, business, and technical applications.

INTRODUCTION

Artificial Intelligence has significantly transformed the way users interact with information systems. Traditional chatbots often rely on predefined rules or large-scale models that may generate inaccurate or irrelevant responses. This limitation becomes critical in domain-specific applications such as education, healthcare, and technical documentation.

To address this issue, the concept of Retrieval-Augmented Generation (RAG) has emerged. RAG enhances language models by combining retrieval mechanisms with generative capabilities. Instead of relying solely on pre-trained knowledge, the system retrieves relevant data from a specific knowledge base and uses it to generate accurate responses.

This project focuses on building a domain-specific chatbot that:

- Uses only provided documents as its knowledge source
- Avoids hallucination through strict prompt control
- Provides accurate and context-based answers

1.PROBLEM STATEMENT

General-purpose AI chatbots often:

- Generate incorrect or fabricated answers
- Lack domain-specific accuracy
- Cannot restrict responses to a defined dataset

The challenge is to design a system that:

- Answers questions strictly from provided documents
- Prevents hallucination
- Maintains contextual accuracy
- Provides an interactive user interface

2.OBJECTIVES

The main objectives of this project are:

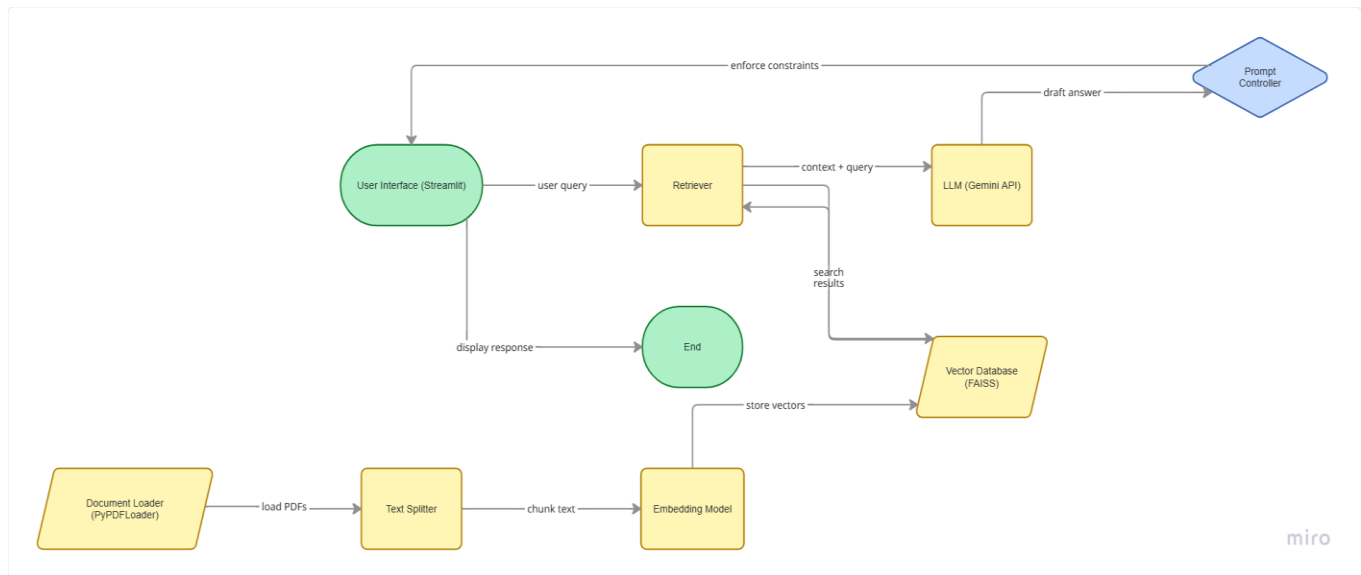
- To develop a domain-specific chatbot using RAG architecture
- To ensure answers are generated only from uploaded PDF documents
- To prevent hallucination using prompt engineering
- To implement efficient document retrieval using FAISS
- To design a user-friendly interface using Streamlit
- To integrate modern AI tools such as LangChain and Gemini API

3.SYSTEM ARCHITECTURE

The system follows a modular architecture consisting of the following components:

1. User Interface (Streamlit)

- Accepts user queries
- Displays chat responses



2. Document Loader

- Loads PDF documents using PyPDFLoader

3. Text Splitter

- Breaks documents into smaller chunks

4. Embedding Model

- Converts text into vector representations

5. Vector Database (FAISS)

- Stores embeddings for efficient retrieval

6. Retriever

- Fetches relevant document chunks

7. LLM (Gemini API)

- Generates responses based on retrieved context

8. Prompt Controller

- Ensures strict answer constraints

4. TECHNOLOGIES USED

- **Python** – Core programming language
- **Streamlit** – Frontend UI for chatbot
- **LangChain** – Framework for building LLM applications
- **Google Gemini API** – Language model for response generation
- **FAISS** – Vector database for similarity search
- **HuggingFace Embeddings** – Text-to-vector conversion
- **PyPDF** – PDF document processing
- **dotenv** – Secure API key management

5. METHODOLOGY

The system follows a structured pipeline:

Step 1: Document Loading

PDF files are loaded using PyPDFLoader from the documents folder or user uploads.

Step 2: Text Splitting

Documents are split into chunks of 500 characters with 50 overlap to maintain context continuity.

Step 3: Embedding Generation

Each chunk is converted into vector embeddings using the HuggingFace model.

Step 4: Vector Storage

Embeddings are stored in FAISS for fast similarity-based retrieval.

Step 5: Query Processing

User query is converted into embedding and compared with stored vectors.

Step 6: Retrieval

Top relevant chunks are retrieved based on similarity.

Step 7: Response Generation

Gemini model generates answer using retrieved context.

Step 8: Prompt Enforcement

Strict rules ensure:

- No external knowledge

- No guessing
- Standard fallback response

6.IMPLEMENTATION

The project is implemented using a modular approach:

- **app.py**: Main application integrating all components
- **documents/**: Stores PDF files
- **test_api.py**: Verifies API key functionality
- **list_model.py**: Checks available Gemini models
- **.env**: Stores API key securely
- **requirements.txt**: Maintains dependency versions

Key features implemented:

- Chat memory using session state
- Dynamic PDF uploads
- Cached vector store for performance
- Strict prompt template for accuracy

Results

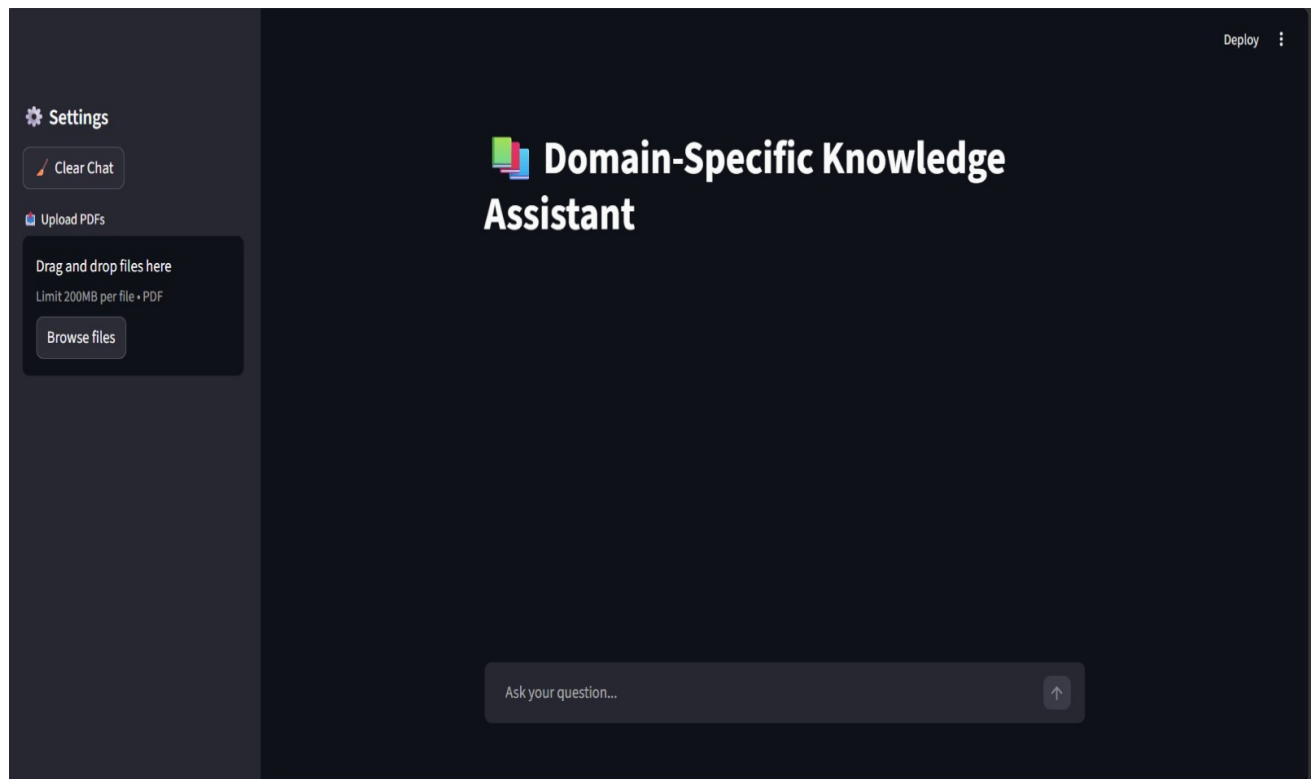
Accurate responses from documents. Invalid queries return a fixed message.

7. RESULTS AND OUTPUT

The chatbot successfully:

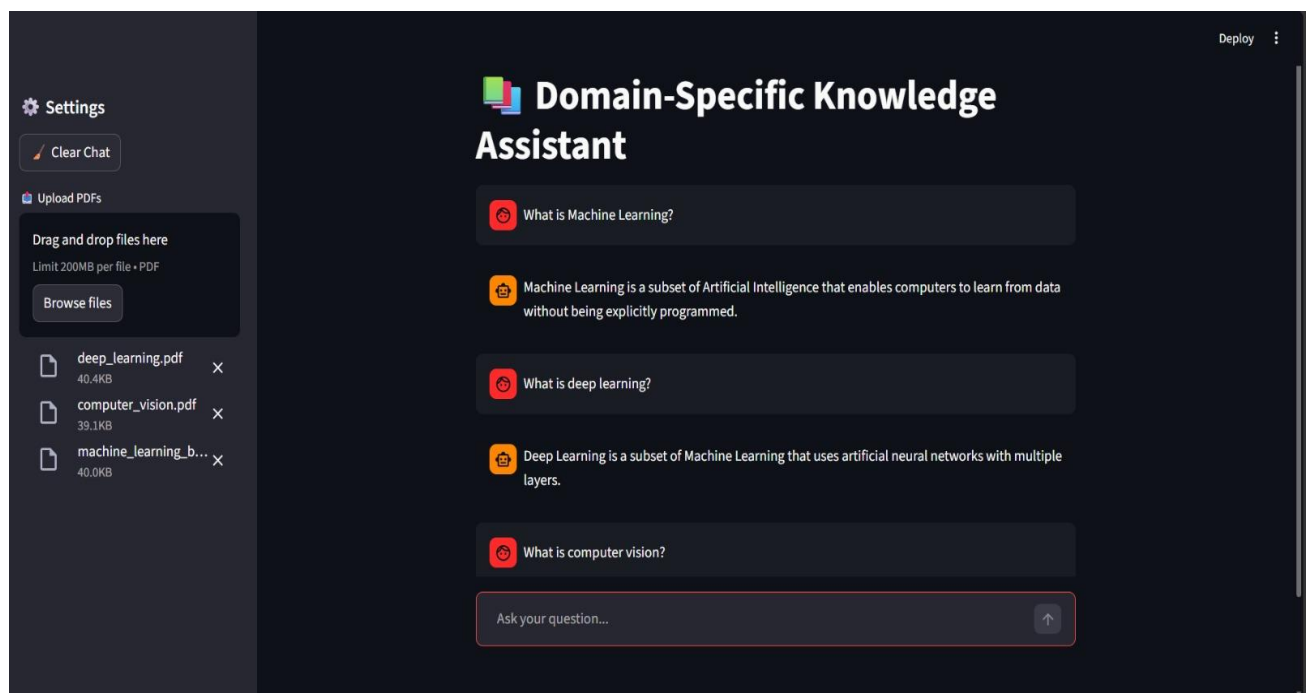
- Answers questions accurately from provided documents
- Rejects unrelated queries with a predefined response
- Maintains conversational flow using chat memory
- Displays responses in a clean chat interface

Figure 1: Initial User Interface



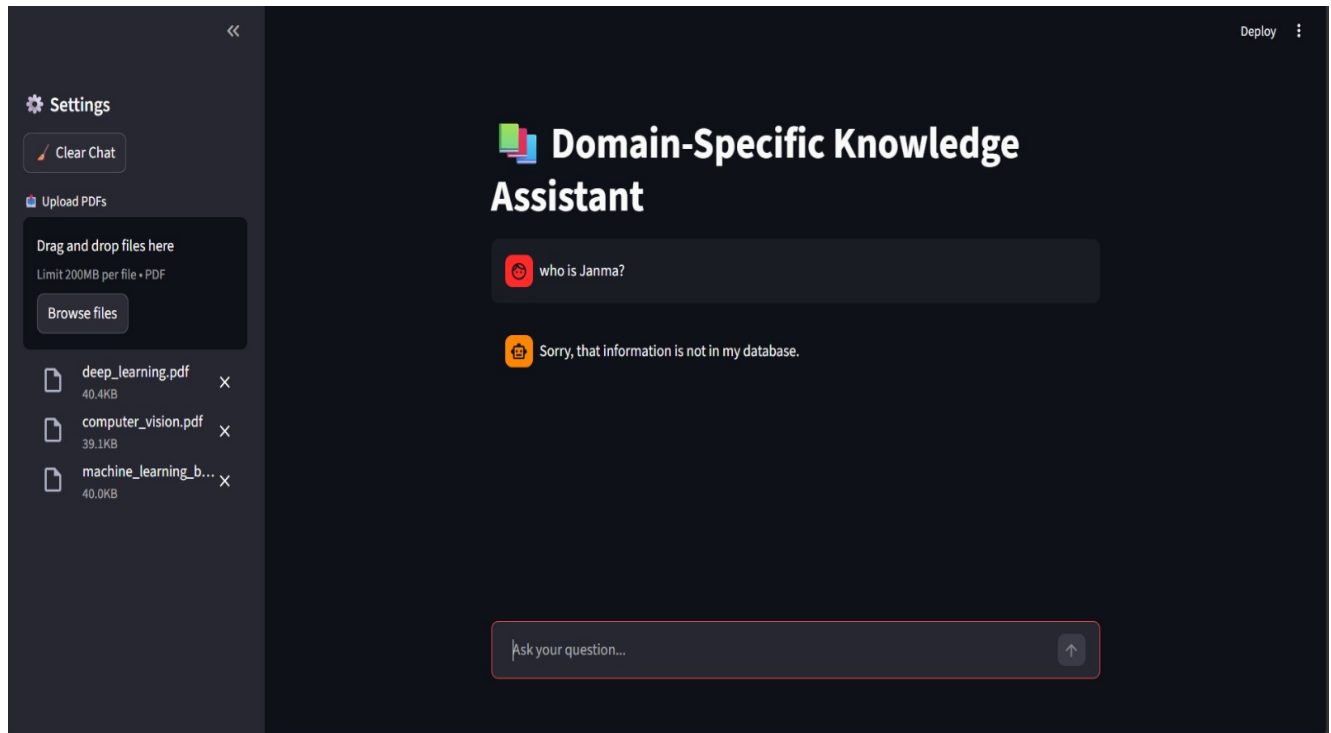
This figure shows the chatbot interface before user interaction, including PDF upload and chat input.

Figure 2: Valid Query Response



This figure demonstrates correct responses for domain-related queries such as Machine Learning and Deep Learning.

Figure 3: Invalid Query Handling



This figure shows how the chatbot handles out-of-scope queries by returning the predefined response, preventing hallucination.

8. ADVANTAGES

- High accuracy due to context-based answering
- Prevents hallucination completely
- Easy to use interface
- Supports multiple document uploads
- Scalable architecture
- Efficient retrieval using FAISS

9. LIMITATIONS

- Works only with provided documents
- Performance depends on document quality
- Limited understanding outside dataset

- Requires API key and internet for LLM
- Not optimized for very large datasets

10. CONCLUSION

The Domain-Specific Knowledge Assistant using RAG successfully demonstrates how modern AI tools can be combined to create a reliable and accurate chatbot. By integrating retrieval mechanisms with generative models, the system ensures that responses are grounded in actual data rather than assumptions.

The use of strict prompt engineering effectively eliminates hallucination, making the system suitable for real-world applications where accuracy is critical. This project highlights the practical implementation of RAG architecture and provides a strong foundation for building advanced AI-powered knowledge systems.